

Bonus Lecture: Numerical Integration

C.Conlon

Fall 2026

- ▶ We are interested in lots of problems that require computing difficult integrals (e.g.: evaluating expectations).
- ▶ Often the problem looks like this:

$$I = \int_a^b h(x) dx$$
$$E_{f|\theta}[g(x)] = \int_a^b g(x) f(x|\theta) dx$$

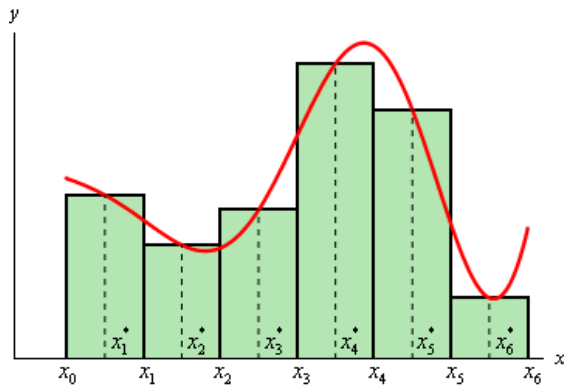
- ▶ Either just integrating $h(x)$ from $[a, b]$ or computing the expectation of $g(x)$ over some density $f(x)$.

- ▶ Our goal is to construct an **integration rule** to approximate the function:

$$\int_a^b f(x) dx \approx \sum_{s=1}^S w_s \cdot f(x_s)$$

- ▶ Rules consist of **nodes** x_s and **weights** w_s
- ▶ This is probably how you learned integration in high school.

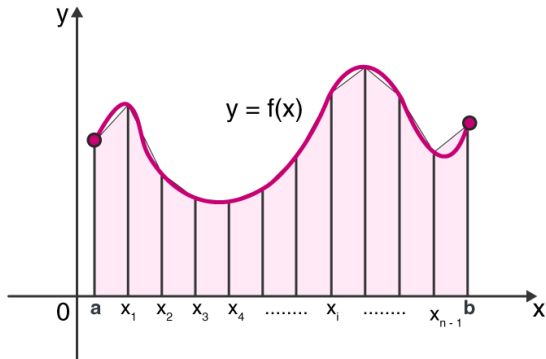
The Midpoint Rule



$$I = \frac{b-a}{S} \cdot \sum_{s=1}^S f(x_s^*) \quad \text{with } x_s^* = \frac{x_{s+1} + x_s}{2}$$

We are fitting **piecewise constants** and adding up **rectangles**

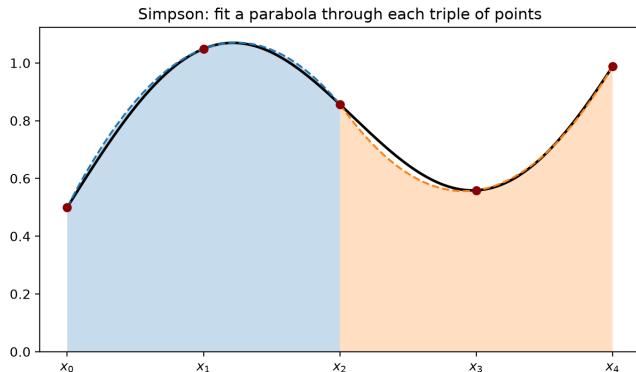
The Trapezoid Rule



$$I = \frac{b-a}{2S} \cdot [f(x_0) + 2 \cdot (f(x_1) + \dots + f(x_{S-1})) + f(x_S)]$$

(1) Fit a (piecewise) line from $f(x_s), f(x_{s+1})$; (2) Evaluate trapezoid area analytically; (3) Add up trapezoids.

Simpson's Rule



$$I = \frac{b-a}{3S} \cdot [f(x_0) + 4(f(x_1) + f(x_3) + \dots) + 2(f(x_2) + f(x_4) + \dots) + f(x_S)]$$

(1) Fit a (piecewise) quadratic through $f(x_s), f(x_{s+1}), f(x_{s+2})$; (2) Analytically integrate the quadratic; (3) Add up areas.

Choosing Integration Rules

Not a lot to choose here

- ▶ Choose the number of points S or interval width $h = \frac{b-a}{S}$.
- ▶ Points are generally equally spaced

How accurate is it?

- ▶ **Bounds analysis** is possible (based on series approximations)
- ▶ For Simpson's Rule:

$$\frac{h^4}{180} (b-a) \max_{\xi \in [a,b]} |f^{(4)}(\xi)|$$

- ▶ Quadratic approximation does poorly where $f^{(4)}$ is large (and gets up to order 3 polynomials exact).

Newton-Cotes Formulas

Summary of formulas for a single interval:

Degree n	Step size h	Common name	Formula	Error term
1	$b - a$	Trapezoid rule	$\frac{h}{2} (f_0 + f_1)$	$-\frac{1}{12} h^3 f^{(2)}(\xi)$
2	$\frac{b-a}{2}$	Simpson's rule	$\frac{h}{3} (f_0 + 4f_1 + f_2)$	$-\frac{1}{90} h^5 f^{(4)}(\xi)$
3	$\frac{b-a}{3}$	Simpson's 3/8 rule	$\frac{3h}{8} (f_0 + 3f_1 + 3f_2 + f_3)$	$-\frac{3}{80} h^5 f^{(4)}(\xi)$
4	$\frac{b-a}{4}$	Boole's rule	$\frac{2h}{45} (7f_0 + 32f_1 + 12f_2 + 32f_3 + 7f_4)$	$-\frac{8}{945} h^7 f^{(6)}(\xi)$

https://en.wikipedia.org/wiki/Newton-Cotes_formulas

Can we do better?

Can use **adaptive rules** (built-in many software routines)

- ▶ Divide up $[a, b]$ evenly into sub-intervals
- ▶ Some are relatively flat or well-approximated, some are not.
- ▶ Use more points in the sub-intervals that are the most difficult to approximate
- ▶ Don't waste points in intervals that are easy to approximate.

Drawbacks

- ▶ Points used in approximation are $f(\cdot)$ dependent
- ▶ Points used in approximation x_s may not be the same for $f(x|\theta)$ at all values of parameters θ .

Gaussian Quadrature: Same Basic Idea

- ▶ Choose a degree of polynomial approximation for $f(x)$ on $[a, b]$.
- ▶ “Fit” the polynomial by evaluating $f(x_s)$ at various values of x_s
- ▶ Integrate the polynomial analytically by adjusting coefficients on $f(x_s)$.

In practice, much easier: get a pre-specified set of weights and nodes (w_s, x_s) .

Gaussian Quadrature: But a little different

- ▶ $[1, x, x^2, x^3, \dots]$ is not the only **polynomial basis**.
- ▶ For example **Chebyshev Basis** (of first kind) is **orthogonal** $[1, x, 2x^2 - 1, 4x^3 - 3x, \dots]$
- ▶ Concept remains the same:
 1. Approximate $f(x)$ with a polynomial basis
 2. Integrate the polynomial exactly
 3. In practice we just get a pre-determined set of points and weights (x_s, w_s) for a given polynomial order.
- ▶ Rules have some additional properties
 - What interval to integrate over $[-1, 1]$ or $[0, \infty)$ or $(-\infty, +\infty)$.
 - Can we exploit properties of $f(x)$: is it proportional to $\frac{1}{\sqrt{1-x^2}}$ or e^{-x} or e^{-x^2} ?
- ▶ May need to do change of variables on $f(x) \rightarrow f(v)$ to better satisfy conditions above.

Gaussian Quadrature

Formulas look like:

$$\int_a^b f(x)dx \approx \sum_{s=1}^S w_s f(x_s)$$

for some quadrature nodes $x_s \in [a, b]$ and weights w_s .

- ▶ Let $\mathcal{F}_k$ be the space of degree k polynomials
- ▶ Quadrature formulas are exact of degree k if it correctly integrates each function in $\mathcal{F}_k$
- ▶ Gaussian quadrature formulas use S points and are exact of degree $2S - 1$.

Approximation Error

$$\int_a^b w(x)f(x)dx - \sum_{s=1}^S w_s f(x_s) = \frac{f^{(2S)}(\xi)}{(2S)!}(p_S, p_S)$$

Legendre Domain: $[-1, 1]$, $w(x) = 1$

Chebyshev Domain: $[-1, 1]$, $w(x) = \frac{1}{\sqrt{1-x^2}}$

Laguerre Domain: $[0, \infty)$, $w(x) = \exp[-x]$ (useful for present value)

Hermite Domain: $(-\infty, \infty)$, $w(x) = \exp[-x^2]$ (useful for normal)

Helpful if function is C^∞ or real-analytic (comports with series expansion).

$$\begin{aligned} E_{f|\theta}[g(x)] &= \int_a^b g(x) f(x|\theta) dx \\ &\approx \frac{1}{S} \sum_{s=1}^S g(x_s) \end{aligned}$$

- ▶ Sample $[x_1, \dots, x_S]$ by drawing from $f(x|\theta)$.
- ▶ Weight everything equally $\frac{1}{S}$
- ▶ Can't bound errors, but can discuss rate of convergence
- ▶ Convergence is slow $\frac{1}{\sqrt{S}}$ but mostly unrelated to curvature of $f(\cdot)$.

Still often want to change variables. Consider $f(x|\theta) \sim N(\mu, \sigma^2)$

- ▶ Should we sample from $x_s \sim N(\mu, \sigma)$?
- ▶ Might be better to sample $z_s \sim N(0, 1)$ and then $x_s = z_s \cdot \sigma + \mu$
- ▶ For complicated distributions $v_s \sim U[0, 1]$ and then $x_s = \Phi^{-1}(v_s) \cdot \sigma + \mu$

Always validate your rule: integrate $f = 1$, $f = x$, $f = x^2$ first — you know those answers, and it tells you which kernel your nodes/weights actually assume.

Example: Gauss-Hermite

Let $Y \sim N(\mu, \sigma^2)$ and apply COV $x = (y - \mu)/\sqrt{2}\sigma$

$$\begin{aligned} E[f(Y)] &= (2\pi\sigma^2)^{-\frac{1}{2}} \int_{-\infty}^{\infty} f(y) \exp\left[-\frac{(y-\mu)^2}{2\sigma^2}\right] dy \\ \int_{-\infty}^{\infty} f(y) \exp\left[-\frac{(y-\mu)^2}{2\sigma^2}\right] dy &= \int_{-\infty}^{\infty} f(\sqrt{2}\sigma x + \mu) e^{-x^2} \sqrt{2}\sigma dx \end{aligned}$$

Gives the quadrature formula using Gauss Hermite (x_s, w_s) .

$$E[f(Y)] = \frac{1}{\sqrt{\pi}} \sum_{s=1}^S w_s f(\sqrt{2}\sigma x_s + \mu)$$

notice that we don't have the e^{-x^2} anymore.

Lots of ways to improve on purely pseudorandom sampling

- ▶ Antithetic draws
- ▶ Low-discrepancy sequences (Halton, Sobol, etc.)
- ▶ Modified Latin Hypercube Sampling (MLHS): Hess, Train, Polak (2006) — a workhorse for mixed logit.
- ▶ Think about these as ensuring more even coverage (stratified).
- ▶ Koksma-Hlawka inequality gives bounds but is hard to characterize the Hardy-Kruse variation of $f(\cdot)$.

Modern home in Python: `scipy.stats.qmc` (Halton, Sobol, LHS); `pyblp.Integration` ships product rules, sparse grids, Halton, and MLHS behind one interface.

- ▶ In higher dimension we can use product rules of 1-D integrals. $\mathbf{x}_s^{(1)} \times \mathbf{x}_s^{(2)}$ and $\mathbf{w}_s^{(1)} \times \mathbf{w}_s^{(2)}$
- ▶ This grows exponentially in dimension D (Curse of Dimensionality)
 - 10 points in dimension one, means 10,000 points in dimension 4.
- ▶ Monte Carlo is not cursed but slow to converge $\frac{1}{\sqrt{S}}$ vs $\frac{1}{2S!} f^{(2S)}$
- ▶ Some monomial rules (Judd), (Skrainka and Judd) aren't cursed
- ▶ Sparse Grids show how to combine 1-D rules more efficiently (next slide)

Idea: instead of the full tensor product, take a clever combination of small tensor products of 1-D rules (Smolyak 1963; for the Gaussian kernel: Heiss and Winschel 2008, <https://www.sparse-grids.de>).

- ▶ A level- K sparse grid is exact for all polynomials of total degree $\leq 2K - 1$ — it gives up the cross terms like $x_1^8 x_2^8$ that the tensor rule pays dearly for.
- ▶ Node counts ($K = 5$): $d = 4$: 385 vs $5^4 = 625$ tensor; $d = 10$: 8,761 vs $5^{10} \approx 9.8$ million.
- ▶ Precomputed nodes/weights for the normal kernel (GQN) are downloadable; or build them yourself from hermgauss via the combination formula.

Learned the hard way (see PS0):

- ▶ Weights are **negative** for many nodes — the rule is not a probability distribution, and cancellation can amplify error when the integrand is far from a low-degree polynomial.
- ▶ Designed for functions dominated by **low-order interactions**; a strongly “diagonal” integrand over a wide range converges slowly.
- ▶ Always run the monomial checks $(1, x_k, x_k^2, x_1^2 x_2^2)$ before trusting any downloaded rule.

Know Your Integrand (Before Brute Force)

The binomial mixed logit from PS0:

$$\mathbb{E}[\Lambda(\beta' X)] \quad \text{with} \quad \beta \sim N(\mu, \text{diag}(\sigma^2)), \quad \beta \in \mathbb{R}^4$$

- ▶ This is a **ridge function**: it varies only along the direction X . And $\beta' X \sim N(\mu' X, \sum_k \sigma_k^2 X_k^2)$ — so the “4-D” integral is **exactly 1-D**.
- ▶ `scipy.integrate.nquad` finds the answer with ~ 10 million evaluations; 1-D quad on the reduced integral needs a few dozen.
- ▶ A tensor rule inherits its 1-D accuracy along the ridge — which is why it can look unbeatable in low d (and still dies at $d = 10$).
- ▶ The multinomial version depends on J indices $\{\beta' x_j\}$, so the reduction is only to $\min(d, J)$ dimensions — the curse is real in BLP-land, but **effective dimension** is often lower than nominal dimension.

How do we draw from a correlated normal?

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} \sim N \left[\begin{pmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \end{pmatrix}, \begin{pmatrix} \sigma_{11}^2 & \sigma_{12}^2 & \sigma_{13}^2 \\ \sigma_{21}^2 & \sigma_{22}^2 & \sigma_{23}^2 \\ \sigma_{31}^2 & \sigma_{32}^2 & \sigma_{33}^2 \end{pmatrix} \right]$$

Or as a vector:

$$\mathbf{Y} \sim N(\boldsymbol{\mu}, \Sigma)$$

1. Find the **Cholesky Root** of $L \cdot L' = \Sigma$ where Σ is $D \times D$ covariance matrix (scipy.linalg.cholesky, chol in R and Matlab).
2. Draw a $n \times D$ dimensional standard normal $\mathbf{Z}$ (either at random or via quadrature nodes/weights).
3. $L \cdot \mathbf{Z} \sim N(0, \Sigma)$, now we have n correlated normals of dimension D .
4. Add back in the mean $\mathbf{Y} = \boldsymbol{\mu} + L \cdot \mathbf{Z} \sim N(\boldsymbol{\mu}, \Sigma)$

Correlated Normals: Why do we do this?

- ▶ L is a $D \times D$ lower triangular matrix
- ▶ Cholesky root has nice properties that guarantees Σ is positive semi-definite.
- ▶ We can get correlated normals just by pre-multiplying by some L matrix.
 - Allows me to draw $\mathbf{Z}$ once
 - re-evaluate at difference covariance matrices Σ using different L .
- ▶ In most cases, optimize over L not Σ .
 - Use delta method to get standard errors for: $L \times L' = \Sigma$
 - In BLP $\mu_{ijt}(\Sigma) = (L \mathbf{Z}_i) \cdot \mathbf{x}_{jt}$ where $(\cdot)$ gives inner product.